REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 1 – TRUE CUSTOMER LIFETIME VALUE (CLV)]

BUSINESS CONTEXT

Brazilian E-Commerce Store is trying to identify their most valuable customers by constructing the True Customer Lifetime Value (CLV). While Customer Lifetime Value (CLV) formulated by total spend as one of the contributing factors as variable, but it is important to also measure how quickly they generate value. For instance, two customers may both spend R$5,000:

- One spends it in their first month
- The other takes 2 years

These two customers are not equally valuable.

Setting The Goal:

Identify Customers with The Highest Lifetime Value and Determine How Long It Takes Each Customer to Reach 50% of Their Total Lifetime Spend.

With This In-Depth Analysis, The Result will help Brazilian E-Commerce:

- Prioritize High-Impact Customers
- Optimize Retention and Loyalty Strategies
- Improve Cash-Flow Forecasting.

HYPOTHESIS

1. Customer Lifetime Value (CLV) Definition
    - CLV = Total Monetary Value of All Payments Made by a Customer
    - Refunds are not explicitly modelled in this dataset
2. Revenue Attribute
    - Revenue is derived from **`order_payments.payment_value`**
    - Each order may have multiple payment records
3. Customer Identity
    - Customers are identified using **`customer_unique_id`**
    - A Customer may place multiple orders over time

4. Time Logic
    - Orders are analysed using **`order_purchase_timestamp`**
    - CLV accumulation follows chronological order of purchases

SQL QUERY

```
WITH order_revenue AS (
    SELECT
        o.order_id,
        c.customer_unique_id,
        o.order_purchase_timestamp,
        SUM(p.payment_value) AS order_revenue
    FROM olist_orders_dataset o
    JOIN olist_customers_dataset c
        ON o.customer_id = c.customer_id
    JOIN olist_order_payments_dataset p
        ON o.order_id = p.order_id
    WHERE o.order_status = 'delivered'
    GROUP BY
        o.order_id,
        c.customer_unique_id,
        o.order_purchase_timestamp
),

customer_cumulative_revenue AS (
    SELECT
        customer_unique_id,
        order_purchase_timestamp,
        order_revenue,
        SUM(order_revenue) OVER (
            PARTITION BY customer_unique_id
            ORDER BY order_purchase_timestamp
        ) AS cumulative_revenue,
        SUM(order_revenue) OVER (
            PARTITION BY customer_unique_id
        ) AS total_clv
    FROM order_revenue
),

clv_50pct_milestone AS (
    SELECT
        customer_unique_id,
        MIN(order_purchase_timestamp) AS date_reached_50pct_clv
    FROM customer_cumulative_revenue
    WHERE cumulative_revenue >= total_clv * 0.5
    GROUP BY customer_unique_id
```

```
)

SELECT
    c.customer_unique_id,
    ROUND(MAX(c.total_clv), 2) AS total_clv,
    MIN(c.order_purchase_timestamp) AS first_purchase_date,
    m.date_reached_50pct_clv,
    TIMESTAMPDIFF(
        DAY,
        MIN(c.order_purchase_timestamp),
        m.date_reached_50pct_clv
    ) AS days_to_50pct_clv
FROM customer_cumulative_revenue c
JOIN clv_50pct_milestone m
    ON c.customer_unique_id = m.customer_unique_id
GROUP BY
    c.customer_unique_id,
    m.date_reached_50pct_clv
ORDER BY total_clv DESC;
```

KEY INSIGHT

Analysis result showing a large proportion of customers are following a pattern of single purchase shopper, meaning they reached 50% of their lifetime spend on the first purchase date. This will result in a biased interpretation of CLV realization speed metrics. Although, a small segment of customers who purchased more frequently, tend to reaches 50% of their lifetime spend within the first 30-45 days and some among them take several months or years. With these findings, it shows that customers who reach 50% Customer Lifetime Value (CLV) quickly are often repeat buyers early, in which they put their purchase in a short time period. Long time-to-50% customers exhibit lower engagement velocity, in particularly long time period, customers tend to have a slow engagement to Customer Lifetime Value.

BUSINESS RECOMMENDATION

1. Customer Segmentation for CLV Interpretation
    - Segment Single-Purchase Customers separately when analysing CLV realization speed
    - Use time-to-50% CLV only within multi-purchase customer cohorts

    - Flag Customers who reach 50% CLV within the first 30 days as high-engagement early repeaters
2. Targeted Retention Campaigns
    - Trigger loyalty incentives after the second or third purchase to reinforce repeat behaviour
    - Offer tailored rewards to frequent early repeat purchasers to sustain momentum
3. Cash Flow and Marketing Efficiency
    - Prioritize Marketing spend on Customer Profiles that historically monetize faster
    - Use early purchase velocity signals to reduce customer acquisition payback periods
4. Predictive Modelling Extension
    - Incorporate early purchase velocity and repeat-purchase timing as features in predictive CLV models
    - Distinguish early repeaters from single-purchase shoppers to improve long-term value forecasting

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 2 – REPEAT PURCHASE BEHAVIOUR]

BUSINESS CONTEXT

Part of the challenges every business is facing is to acquire or gain new customers. Brazilian E-Commerce Store wants to understand how quickly customers come back after their first purchase. Identifying these patterns will give a starting point for businesses to learn Customer Behaviour more in-depth.

Setting The Goal:

How many customers make a second purchase within 30, 60, and 90 days of their first purchase order.

With This In-Depth Analysis, The Result will help Brazilian E-Commerce:

- Measure early customer engagement
- Optimize onboarding and remarketing campaigns
- Improve Customer Lifetime Value (CLV)

HYPOTHESIS

1. Repeat Purchase Definition
    - A Repeat Purchase is defined as a second delivered order
    - The second order must occur after the first purchase date
2. Customer Identity
    - Customers are identified using **`customer_unique_id`**
3. Order Validity
    - Only orders with **`order_status = 'delivered'`** are considered
4. Time Measurement
    - Time between purchases is measured using **`order_purchase_timestamp`**
    - Calculations are done in days.

SQL QUERY

```
WITH ranked_orders AS (
    SELECT
        c.customer_unique_id,
        o.order_purchase_timestamp,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_unique_id
            ORDER BY o.order_purchase_timestamp
        ) AS order_rank
    FROM olist_orders_dataset o
    JOIN olist_customers_dataset c
        ON o.customer_id = c.customer_id
    WHERE o.order_status = 'delivered'
),

first_second_purchase AS (
    SELECT
        customer_unique_id,
        MAX(CASE WHEN order_rank = 1 THEN order_purchase_timestamp
END) AS first_purchase_date,
        MAX(CASE WHEN order_rank = 2 THEN order_purchase_timestamp
END) AS second_purchase_date
    FROM ranked_orders
    GROUP BY customer_unique_id
)

SELECT
    COUNT(*) AS total_customers,

    SUM(
        CASE
            WHEN second_purchase_date IS NOT NULL
             AND       TIMESTAMPDIFF(DAY,       first_purchase_date,
second_purchase_date) <= 30
            THEN 1 ELSE 0
        END
    ) AS repeat_within_30_days,

    ROUND(
        100.0 * SUM(
            CASE
                WHEN second_purchase_date IS NOT NULL
                 AND     TIMESTAMPDIFF(DAY,     first_purchase_date,
second_purchase_date) <= 30
                THEN 1 ELSE 0
            END
        ) / COUNT(*), 2
```

```
    ) AS pct_repeat_30_days,

    SUM(
        CASE
            WHEN second_purchase_date IS NOT NULL
             AND       TIMESTAMPDIFF(DAY,       first_purchase_date,
second_purchase_date) <= 60
            THEN 1 ELSE 0
        END
    ) AS repeat_within_60_days,

    ROUND(
        100.0 * SUM(
            CASE
                WHEN second_purchase_date IS NOT NULL
                 AND     TIMESTAMPDIFF(DAY,     first_purchase_date,
second_purchase_date) <= 60
                THEN 1 ELSE 0
            END
        ) / COUNT(*), 2
    ) AS pct_repeat_60_days,

    SUM(
        CASE
            WHEN second_purchase_date IS NOT NULL
             AND       TIMESTAMPDIFF(DAY,       first_purchase_date,
second_purchase_date) <= 90
            THEN 1 ELSE 0
        END
    ) AS repeat_within_90_days,

    ROUND(
        100.0 * SUM(
            CASE
                WHEN second_purchase_date IS NOT NULL
                 AND     TIMESTAMPDIFF(DAY,     first_purchase_date,
second_purchase_date) <= 90
                THEN 1 ELSE 0
            END
        ) / COUNT(*), 2
    ) AS pct_repeat_90_days

FROM first_second_purchase;
```

KEY INSIGHT

Repeat Purchase Behaviour Analysis shows that only a small percentage of Customers return for a second purchase within 30 days. The majority of repeat purchases happen within 60 – 90 days, not immediately. With 1.52% customers are returning within 30 days, 1.84% customers are returning within 60 days, and 2.05% customers are returning within 90 days. Meaning it would be considered as low penetration of customers are likely to repeat purchase even within the range of 90 days. Customers who do not return within 90 days are unlikely to become repeat buyers.

BUSINESS RECOMMENDATION

1. 30-Day Activation Campaign
    - Trigger discounts or reminders 7 – 14 days after first purchase. A short window period of time will likely motivate customers to use or benefited the free additional promos after first purchase. Making customer feels seen.
    - Focus on moving customers into the "30-day repeat" bucket. The more engaging and easier to access customers are the ones who are in a repeat purchase pattern. Main focus is to create a repeat purchase pattern into 30-days window.
2. 60 – 90 Day Re-Engagement
    - Use email or push notifications for customers approaching 60 days without a second purchase. Customers might forget about the first purchase they order. It will help to remind customers to purchase their order again.
3. Customer Lifetime Value (CLV) Integration
    - Combine repeat timing with CLV velocity. With the resulted of small segment and large segment of customers in the CLV velocity, it would gain more substantial findings to connect how long the customers are return within 30, 60, and 90 days window.
4. Marketing ROI Optimization
    - Reduce spend on customers with no repeat activity after 90 days.
    - Shift budget toward early reactivation strategies.

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 3 – ORDER FUNNEL DROP-OFF ANALYSIS]

BUSINESS CONTEXT

Brazilian E-Commerce Store would like to understand the penetration of customers are likely to churn or discontinue their Customer Journey until purchased orders are delivered. Brazilian E-Commerce operates a multi-step order lifecycle. Although, not all orders successfully reach delivery.

Setting The Goal:
At Which Stage Do We Lose The Most Orders: Payment Approval, Seller Shipment, Or Final Delivery?

Discover the result will help Brazilian E-Commerce:

- Reduce Operational Friction
- Improve Seller Performance
- Increase Completed (Delivered) Orders
- Improve Revenue Realization

HYPOTHESIS

1. Order Funnel Stages
   The Brazilian E-Commerce Store Order Lifecycle is modelled as:
   - Order Placed
   - Payment Approved
   - Shipped By Seller
   - Delivered To Customer
2. Stage Definitions (Data-Driven)
   - Order Placed: All records in `olist_orders_dataset`
   - Payment Approved: `order_approved_at IS NOT NULL`
   - Shipped: `order_delivered_carrier_date IS NOT NULL`
   - Delivered: `order_delivered_customer_date IS NOT NULL`
3. One Order = One Funnel

- Each order progresses sequentially through stages
- Orders may drop off at any stage

SQL QUERY

```
WITH order_funnel AS (
    SELECT
        order_id,

        CASE
            WHEN                    order_status                    IN
('approved','processing','shipped','delivered')
            THEN 1 ELSE 0
        END AS payment_approved,

        CASE
            WHEN order_status IN ('shipped','delivered')
            THEN 1 ELSE 0
        END AS shipped,

        CASE
            WHEN order_status = 'delivered'
            THEN 1 ELSE 0
        END AS delivered

    FROM olist_orders_dataset
)

SELECT
    COUNT(*) AS total_orders,

    SUM(payment_approved) AS payment_approved_orders,
    SUM(shipped) AS shipped_orders,
    SUM(delivered) AS delivered_orders,

    ROUND(
        100.0 * SUM(payment_approved) / COUNT(*), 2
    ) AS pct_order_to_payment,

    ROUND(
        100.0 * SUM(shipped) / SUM(payment_approved), 2
    ) AS pct_payment_to_shipped,

    ROUND(
        100.0 * SUM(delivered) / SUM(shipped), 2
    ) AS pct_shipped_to_delivered
```

```
FROM order_funnel;
```

KEY INSIGHT

Analysis result showing that the lowest conversion rate extracted from order to payment. About 98.44% of customers placed orders progress to the payment approved stage, indicating low friction during checkout and effective payment methods and authorization flow. There is also a minority noticeable drop-off between payment approval and shipment, suggesting operational bottlenecks after payment is captured and potential delays in seller confirmation, inventory allocation, or logistics handoff. While the conversion to delivered are also high, meaning once shipped, orders are very likely to be delivered implying reliable last-mile logistics and few losses or cancellations after shipment begins. The order funnel shows that the greatest execution risk exists after payment but before shipment, not during checkout or delivery.

BUSINESS RECOMMENDATION

1. Prioritize Seller Fulfilment SLAs After Payment Approval
    - Introduce or tighten SLAs for sellers to confirm and ship orders promptly
    - Offer a various payment methods to customers for an easy and accessible payment steps
    - Monitor seller-level shipment latency and penalize repeated delays
2. Improve Inventory Availability and Reservation Logic
    - Ensure stock is effectively reserved at payment approval
    - Reduce post-payment cancellation caused by inventory mismatches
3. Proactive Customer Communication During Fulfilment Lag
    - Send automated updates if shipment does not occur within particular hours after payment
    - Transparency reduces cancellations and negative reviews
4. Create an Operational KPI of Payment-To-Shipment Time
    - Track median time from payment approval to shipment and use this KPI to identify underperforming sellers and optimize warehouse and logistics workflows.

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 4 – LATE DELIVERY IMPACT ON REVIEWS]

BUSINESS CONTEXT

Customer Satisfactions are measures on particular contribution factors, one of the factors are derived from Delivery Time. Customer Reviews are critical driver of Trust, Conversion, and Seller Reputation in an E-Commerce Marketplaces. A 5-Star Reviews an a Positive Feedbacks are a perfect foundation of an Online Marketplace Seller. Late deliveries are one of the most common operational failures and are suspected to strongly influence Customer Dissatisfaction.

Setting The Goal:

Quantify How Late Delivery Affects Review Scores And Identify The Magnitude Of Customer Sentiment Damage Caused By Delivery Delays.

HYPOTHESIS

1. An order is considered late if
   order_delivered_customer_date > order_estimated_delivery_date
2. Only delivered orders are analysed (reviews before delivery are excluded)
3. Each order has at most one review in **`olist_order_reviews_dataset`**
4. Review scores range from 1 (worst) to 5 (best).

SQL QUERY

```
WITH delivery_performance AS (
     SELECT     order_id,
                CASE     WHEN     order_delivered_customer_date     >
order_estimated_delivery_date THEN 1 ELSE 0 END AS is_late
    FROM olist_orders_dataset
    WHERE order_status = 'delivered'
),

review_analysis AS (
     SELECT     dp.is_late,
                r.review_score
    FROM delivery_performance dp
```

```
    JOIN olist_order_reviews_dataset r
           ON dp.order_id = r.order_id
)

SELECT     is_late,
           COUNT(*) AS total_orders,
        ROUND(AVG(review_score), 2) AS avg_review_score,
        SUM(CASE WHEN review_score <= 2 THEN 1 ELSE 0 END) AS
low_score_orders,
        ROUND(100.0 * SUM(CASE WHEN review_score <= 2 THEN 1 ELSE 0
END)/COUNT(*), 2) AS pct_low_reviews
FROM review_analysis
GROUP BY is_late
```

KEY INSIGHT

Approaching Customer Satisfaction Improvements, the analysis shows that in fact late deliveries highly impacted the business standings. Late deliveries receive significantly lower review scores. Orders delivered after the estimated date show a material drop in average review score compared to on-time deliveries. Late orders are disproportionately responsible for 1–2 star reviews, the percentage of low ratings (≤2 starts) is substantially higher for late deliveries, about more than a half customer who received the order late than the estimated date are growing concerns and eventually resulted in low score of reviews. This confirms that delivery delays as a primary driver of negative feedbacks. Customers tend to punish lateness as late delivery resulted in poor review while on-time delivery resulted in neutral to positive review. Operational delays have reputational consequences since review scores affect seller rankings, conversion rates, and trust in the platform.

BUSINESS RECOMMENDATION

1. Treat Delivery Delays as a Reputational Risk, Not Just an Operational Metrics
    - Incorporate lateness penalties into seller performance scoring
    - Surface delivery reliability in seller front dashboards
2. Set Tighter Estimated Delivery Dates
    - Conservative delivery estimates reduce expectation violations
    - Even small buffers can significantly improve review outcomes
3. Proactive Customer Communication for At-Risk Orders

- Notify customers when delays are detected early
- Offer compensation such as credits or vouchers before dissatisfaction escalates

4. Target High-Impact Fixes
    - Focus delay reduction on high-volume sellers or product categories with historically low delivery reliability.

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 5 – SELLER REVENUE STABILITY ANALYSIS]

BUSINESS CONTEXT

Revenue is not about the profit and platform reputability, Marketplace Store who have a stable revenue tends to track their profit income predictable and a steady income with off-risk probabilities. In a marketplace, not all revenue is equally reliable. Some sellers generate consistent, predictable revenue, while others are highly volatile creating risk for platform forecasting, seller churn, and operational planning.

Setting The Goal:

Identify Revenue Stability At The Seller Level By Measuring Revenue Concentration, Revenue Volatility Over Time, and Dependency Risk On Top Sellers.

HYPOTHESIS

1. Revenue is calculated from **`olist_order_items_dataset`** by:
   revenue = **`price`** + **`freight_value`**
2. Only delivered orders are considered realized revenue.
3. Revenue stability is evaluated monthly.
4. Sellers with very low order counts are excluded to avoid noise.

SQL QUERY

```
WITH seller_monthly_revenue AS (
     SELECT     oi.seller_id,
                DATE_FORMAT(o.order_purchase_timestamp,  '%Y-%m')  AS
revenue_month,
            SUM(oi.price + oi.freight_value) AS monthly_revenue,
            COUNT(DISTINCT o.order_id) AS total_orders
    FROM olist_order_items_dataset oi
    JOIN olist_orders_dataset o
           ON oi.order_id = o.order_id
     WHERE o.order_status = 'delivered'
```

```
    GROUP BY oi.seller_id, revenue_month
),

filtered_sellers AS (
      SELECT seller_id
    FROM seller_monthly_revenue
    GROUP BY seller_id
    HAVING COUNT(DISTINCT revenue_month) >= 3
)

SELECT     smr.seller_id,
           ROUND(AVG(smr.monthly_revenue), 2) AS avg_monthly_revenue,
        ROUND(STDDEV(smr.monthly_revenue), 2) AS revenue_stddev,

ROUND(STDDEV(smr.monthly_revenue)/NULLIF(AVG(smr.monthly_revenue),
0), 2) AS revenue_volatility_ratio,
        COUNT(DISTINCT revenue_month) AS active_months
FROM seller_monthly_revenue smr
JOIN filtered_sellers fs
      ON smr.seller_id = fs.seller_id
GROUP BY smr.seller_id
ORDER BY revenue_volatility_ratio DESC
```

KEY INSIGHT

A small subset of sellers shows extreme revenue volatility. Some sellers have a high average revenue but also large month-to-month fluctuations, indicating seasonal dependence, promotion-driven spikes, and unstable inventory pipelines. Revenue concentration risk exists at the seller level which sellers with high average revenue and high volatility ratio pose a forecasting and dependency risk for the platform. Consistent sellers are more valuable than high-revenue volatile sellers. Sellers with moderate revenue but low volatility ratios provide predictable cash flow and lower operational and support burden. Seller maturity correlates with stability. Sellers active across more months tend to have lower volatility, suggesting experience and operational maturity reduce revenue risk.

BUSINESS RECOMMENDATION

1. Segment Sellers by Revenue Stability
    - Stable Seller → Preferred partnerships and growth incentives
    - Volatile Sellers → Monitoring, operational support, or, risk controls

2. Reduce Platform Dependency on Volatile Top Sellers
    - Diversify seller revenue sources
    - Encourage growth among stable mid-tier sellers
3. Use Stability Metrics in Seller Health Scoring
    - Introduce Revenue Volatility Ratio as a core KPI
    - Combine with delivery performance and review score
4. Target Enablement for High-Volatility Sellers
    - Inventory planning support
    - Logistics optimization
    - Promotional calendar smoothing

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 6 – SELLER SLA COMPLIANCE]

BUSINESS CONTEXT

In Sellers market, regulation towards platform will show a report of underperforming and overperforming Sellers. Sellers are expected to ship orders within a defined Service Level Agreement (SLA). Failure to comply leads to delivery delays, poor customer customers, and platform trust erosion

Setting The Goal:
Measure Seller Compliance with Shipment SLAs, Identify Underperforming Sellers and Quantify Platform Risk.

HYPOTHESIS

1. SLA Definition
   A Seller is considered SLA-Compliant if
   `order_delivered_carrier_date <= order_estimated_delivery_date`
   (Shipment handed to carrier on or before the promised delivery date)
2. Only delivered orders are analysed
3. SLA performance is measured at the Seller level
4. Sellers with very few orders are excluded to reduce noise.

SQL QUERY

```
WITH seller_sla AS (
     SELECT     oi.seller_id,
                o.order_id,
            CASE     WHEN     o.order_delivered_carrier_date     <=
o.order_estimated_delivery_date THEN 1 ELSE 0 END AS sla_compliant
    FROM olist_order_items_dataset oi
    JOIN olist_orders_dataset o
           ON oi.order_id = o.order_id
     WHERE o.order_status = 'delivered'
),

seller_sla_summary AS (
```

```
      SELECT     seller_id,
                 COUNT(DISTINCT order_id) AS total_orders,
            SUM(sla_compliant) AS sla_compliant_orders
    FROM seller_sla
    GROUP BY seller_id
)

SELECT     seller_id,
           total_orders,
        sla_compliant_orders,
        ROUND(100.0   *   sla_compliant_orders/total_orders,   2)   AS
sla_compliance_rate
FROM seller_sla_summary
WHERE total_orders >= 20
ORDER BY sla_compliance_rate ASC
```

KEY INSIGHT

Analysis result showing a small group of Sellers creates a low SLA Compliance Rate regarding late deliveries which delivery carrier date later then the estimated delivery date. Although, the results captured about 2 sellers only showing an SLA Compliance Rate lower than 80%. SLA Compliance varies widely across sellers. While many sellers maintain high compliance rates, a significant subset consistently fails to meet shipment SLAs. Low-Compliance Sellers pose disproportionate risk. Sellers with poor SLA Compliance are likely contributors to late deliveries, low review scores, and customer churn. Volume does not guarantee compliance. Some high-volume sellers still exhibit weak SLA performance, indicating operational strain, poor logistics coordination, and scaling issues. SLA Compliance is a leading indicator of Customer Experience. Poor compliance often precedes late delivery flags, negative reviews, and increased customer support tickets.

BUSINESS RECOMMENDATION

1. Introduce SLA-Based Seller Tiering
    - High Compliance → Preferred Seller status, fee discounts
    - Low Compliance → Monitoring, penalties, or restrictions
2. Enforce Minimum SLA Thresholds
    - Require Sellers to maintain ≥ 95% compliance
    - Trigger intervention workflows below thresholds

3. Operational Support for At-Risk Sellers
    - Logistics onboarding
    - Carrier partnerships
    - Shipment automation tools
4. Integrate SLA Metrics into Seller Health Score

   Combine into a single seller performance index that includes:
    - SLA Compliance
    - Review Score
    - Revenue Stability

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 7 – CATEGORY PROFITABILITY vs VOLUME]

BUSINESS CONTEXT

Brazilian E-Commerce Store would be interested to analyse Product Category Profitability compared to sales volume. High sales volume does not always imply high profitability. Some product categories generate large order volumes but thin margins, and other have low volume but high-value transactions.

Setting The Goal:

Evaluate Product categories Across Volume and Revenue Dimensions To Identify High-Volume/ Low-Profit Categories, Low-Volume/ High-Profit Categories, and Strategic Focus Areas for Growth and Optimization.

HYPOTHESIS

1. Revenue is calculated as:
   `price + freight_value`
2. Profit is approximated using Gross Revenue
3. Category Names are translated using
   `product_category_name_translation`
4. Only delivered orders are included.

SQL QUERY

```
WITH category_sales AS (
     SELECT     pt.product_category_name_english AS category,
                COUNT(DISTINCT o.order_id) AS total_orders,
            COUNT(oi.order_item_id) AS total_items,
            SUM(oi.price + oi.freight_value) AS total_revenue,
            ROUND(100.0   *   SUM(oi.price   +   oi.freight_value)/
COUNT(DISTINCT o.order_id), 2) AS avg_order_value
    FROM olist_order_items_dataset oi
    JOIN olist_orders_dataset o
           ON oi.order_id = o.order_id
      JOIN olist_products_dataset p
           ON oi.product_id = p.product_id
```

```
      JOIN product_category_name_translation pt
           ON p.product_category_name = pt.product_category_name
      WHERE o.order_status = 'delivered'
    GROUP BY pt.product_category_name_english
)

SELECT     category,
           total_orders,
        total_items,
        ROUND(total_revenue, 2) AS total_revenue,
        avg_order_value,

        NTILE(4) OVER (ORDER BY total_orders DESC) AS volume_quartile,
        NTILE(4)   OVER   (ORDER   BY   total_revenue   DESC)   AS
revenue_quartile
FROM category_sales
ORDER BY total_revenue DESC
```

KEY INSIGHT

Analysis showing that high volume does not guarantee high revenue. Several categories rank in the top volume quartile but fall into lower revenue quartiles including low price points, thin margins, and high operational burden per dollar earned. Low-Volume, High-Revenue Categories exists. Some categories generate significant revenue despite fewer orders, suggesting premium pricing, higher-value items, and upsell or bundling potential. Operational cost may be misaligned with revenue contribution. Categories with high order counts but low average order value likely consume disproportionate logistics and support resources, and deliver lower ROI per transaction. Category portfolio is unbalanced. Revenue concentration in a few categories exposes the platform to demand shocks, seasonality risk, and supplier dependency.

BUSINESS RECOMMENDATION

1. Reevaluate Investment in High-Volume, Low-Revenue Categories
    - Optimize fulfilment costs
    - Introduce minimum order thresholds
    - Bundle low-value items
2. Double Down on High-Revenue Categories
    - Improve discovery and promotion
    - Attract premium sellers

    - Protect SLA and quality standards
3. Introduce Category-Specific Strategies
    - Different commission rates by category
    - Customized logistics models
    - Targeted marketing campaigns
4. Monitor Category Health via a 2x2 Matrix
    - Volume vs Revenue Quadrant Tracking
    - Flag Categories drifting into low-ROI territory

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 8 – CATEGORY SEASONALITY DETECTION]

BUSINESS CONTEXT

As many E-Commerce faces a trend seasonal profit, E-Commerce demand often fluctuates by season. Certain product categories experience holiday spikes, annual cycles, and periodic drops. Understanding seasonality at the category level allows platform to:

- Optimize inventory planning
- Schedule promotions
- Prepare logistics capacity
- Improve revenue forecasting

Setting The Goal:

Detect Seasonal Demand Patterns By Product Category Using Historical Order Data.

HYPOTHESIS

1. Seasonality is measured using monthly order volume
2. Only delivered orders are included
3. Product categories are mapped via: **`product_category_name_translation`**
4. Seasonality is inferred by **consistent monthly deviations from a category average demand**.

SQL QUERY

MONTHLY CATEGORY DEMAND

```
WITH category_monthly_orders AS (
     SELECT     pt.product_category_name_english AS category,
                DATE_FORMAT(o.order_purchase_timestamp,  '%Y-%m')  AS
year_months,
            COUNT(DISTINCT o.order_id) AS monthly_orders
```

```
    FROM olist_orders_dataset o
    JOIN olist_order_items_dataset oi
           ON o.order_id = oi.order_id
      JOIN olist_products_dataset p
           ON oi.product_id = p.product_id
      JOIN product_category_name_translation pt
           ON p.product_category_name = pt.product_category_name
      WHERE o.order_status = 'delivered'
    GROUP            BY            pt.product_category_name_english,
DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')
)

SELECT *
FROM category_monthly_orders
ORDER BY category, year_months
```

SEASONALITY INDDEX BY CATEGORY AND MONTH

```
WITH category_monthly_orders AS (
      SELECT     pt.product_category_name_english AS category,
                 MONTH(o.order_purchase_timestamp) AS monthly_num,
             COUNT(DISTINCT o.order_id) AS monthly_orders
    FROM olist_orders_dataset o
    JOIN olist_order_items_dataset oi
           ON o.order_id = oi.order_id
      JOIN olist_products_dataset p
           ON oi.product_id = p.product_id
      JOIN product_category_name_translation pt
           ON p.product_category_name = pt.product_category_name
      WHERE o.order_status = 'delivered'
    GROUP            BY            pt.product_category_name_english,
MONTH(o.order_purchase_timestamp)
),

category_avg AS (
      SELECT     category,
                 AVG(monthly_orders) AS avg_monthly_orders
    FROM category_monthly_orders
    GROUP BY category
)

SELECT     cmo.category,
           cmo.monthly_num,
        cmo.monthly_orders,
        ROUND(ca.avg_monthly_orders, 2) AS avg_monthly_orders,
        ROUND(cmo.monthly_orders/  NULLIF(ca.avg_monthly_orders,  0),
2) AS seasonality_index
FROM category_monthly_orders cmo
```

```
JOIN category_avg ca
     ON cmo.category = ca.category
ORDER BY cmo.category, cmo.monthly_num
```

KEY INSIGHT

Analysis showing that clear seasonal peaks exist in multiple categories. Several categories exhibit consistent demand spikes in specific months, indicating strong seasonality rather than random variation. Although, not all categories are seasonal. Some categories maintain relatively stable demand throughout the year, making them ideal for baseline revenue forecasting. Seasonality is category-specific. Peak months differ across categories, meaning platform-wide promotions may be inefficient if not targeted. Operational load is predictable. Seasonal spikes allow proactive planning for inventory allocation, carrier capacity, and seller readiness. Overall Seasonality Index are categorized by these definitions:

Seasonality Index	Category
> 1.2	Strong Seasonal Peak
0.9 – 1.2	Normal Demand
0.5 – 0.8	Seasonal Low
<0.5	Severe Demand Drop

With 1.5 seasonality Index is considered 50% above normal and 0.6 considered 40% below normal.

BUSINESS RECOMMENDATION

1. Adopt Category-Specific Promotion Calendars
    - Promote seasonal categories during peak months
    - Avoid discounting during naturally high-demand periods
2. Align Inventory & Logistics with Seasonal Signals
    - Pre-stock fast-moving seasonal categories
    - Scale logistics capacity ahead of expected peaks
3. Improve Forecasting Accuracy
    - Incorporate seasonality indices into demand forecasting models

    - Reduce stockout and overstock risk
4. Seller Enablement for Seasonal Readiness
    - Notify sellers of upcoming seasonal peaks
    - Encourage early inventory preparation

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 9 – INSTALLMENT RISK ANALYSIS]

BUSINESS CONTEXT

Installments is one of the provided payments from Brazilian E-Commerce Store. Many E-Commerce platforms allow customers to pay using installments. While installments can increase conversion, they also introduce financial and operational risk, such as:

- Higher cancellation rates
- Increased defaults or disputes
- Cash flow delays

Setting The Goal:

Analyse How Installment Usage Affects Order Outcomes and Customer Behaviour, and Identify Risk Signals Associated with Installment-Heavy Orders.

HYPOTHESIS

1. Installment information comes from **`olist_order_payments_dataset`**
2. An Order is considered high installment if **`payment_installments > 1`**
3. Only delivered orders are included
4. Each order may have multiple payment records (Aggregated at Order Level)

SQL QUERY

INSTALLMENT BEHAVIOUR

```
WITH order_installments AS (
     SELECT     order_id,
                MAX(payment_installments) AS installments
    FROM olist_order_payments_dataset
    GROUP BY order_id
),
```

```
order_outcomes AS (
     SELECT     o.order_id,
                oi.installments,
                CASE WHEN oi.installments > 1 THEN 'installment' ELSE
'single_payment' END AS payment_type
    FROM olist_orders_dataset o
    JOIN order_installments oi
           ON o.order_id = oi.order_id
     WHERE o.order_status = 'delivered'
)

SELECT     payment_type,
           COUNT(*) AS total_orders,
        ROUND(AVG(installments), 2) AS avg_installments
FROM order_outcomes
GROUP BY payment_type
```

INSTALLMENTS vs REVIEW RISK

```
WITH order_installments AS (
     SELECT     order_id,
                MAX(payment_installments) AS installments
    FROM olist_order_payments_dataset
    GROUP BY order_id
)

SELECT     CASE WHEN oi.installments > 1 THEN 'installment' ELSE
'single_payment' END AS payment_type,
           COUNT(*) AS total_orders,
        ROUND(AVG(r.review_score), 2) AS avg_review_score,
        ROUND(100.0 * SUM(CASE WHEN r.review_score <= 2 THEN 1 ELSE 0
END)/ COUNT(*), 2) AS pct_low_reviews
FROM olist_orders_dataset o
JOIN order_installments oi
     ON o.order_id = oi.order_id
JOIN olist_order_reviews_dataset r
     ON o.order_id = r.order_id
WHERE o.order_status = 'delivered'
GROUP BY payment_type
```

KEY INSIGHT

Installment Analysis shows that Installment orders represent a meaningful share of transactions. A significant portion of delivered orders use installment payments, confirming their importance for conversion. Installment usage correlates with lower review scores. Orders paid through installments tend to have slightly lower average review scores and higher proportions of low (1-2 star) reviews. Installments amplify dissatisfaction when issues occur. Customers paying over time are more sensitive to delivery delays and product quality issues since dissatisfaction persist across multiple billing cycles. Installments introduce latent platform risk. Even if delivery succeeds, installment-heavy orders tend to create delay cash realization, increase dispute exposure, and require stronger post purchase support.

BUSINESS RECOMMENDATION

1. Introduce Installment Risk Monitoring
    - Track review score and dispute rates by installment count
    - Flag high-risk installment patterns early
2. Limit Installments for High-Risk Segments
    - Cap installment counts for New Customers, Low-Rated Sellers, and High-Return Categories
3. Align Installments with Operational Reliability
    - Offer longer installment plans only to High-SLA Sellers and Categories with String Delivery Performance
4. Use Installments as a Strategic Lever
    - Encourage installments where reviews are consistently high and deliver reliability is proven
    - Avoid blanket installment availability

REPORT ANALYSIS OF BRAZILIAN E-COMMERCE
[ANALYSIS 10 – ABNORMAL PAYMENT PATTERNS DETECTION]

BUSINESS CONTEXT

E-Commerce platforms must detect abnormal payment behaviour to mitigate:

- Fraud Risk
- Payment Disputes
- Chargebacks
- Operational Anomalies

Abnormal payment patterns may include

- Excessive installment counts
- Unusual payment amounts
- Multiple payments per order
- Payment behaviour inconsistent with order value

Setting The Goal:
Identify Orders with Suspicious or Abnormal Payment Characteristics Using Payment-Level Data.

HYPOTHESIS

1. Payment data comes from **`olist_order_payments_dataset`**
2. An order may have multiple payment records
3. Abnormal behaviour is defined using statistical and logical thresholds, not labels
4. Only delivered orders are analysed to avoid noise from cancellations.

SQL QUERY

ORDER-LEVEL PAYMENT PROFILE

```
WITH order_payment_profile AS (
          SELECT     p.order_id,
                     COUNT(*) AS payment_attempts,
```

```
                SUM(p.payment_value) AS total_payment_value,
                MAX(p.payment_installments) AS max_installments,
                COUNT(DISTINCT p.payment_type) AS payment_type_count
        FROM olist_order_payments_dataset p
        GROUP BY p.order_id
),

order_value AS (
     SELECT     order_id,
                SUM(price + freight_value) AS order_value
    FROM olist_order_items_dataset
    GROUP BY order_id
)

SELECT     o.order_id,
           opp.payment_attempts,
        opp.total_payment_value,
        ROUND(ov.order_value, 2) AS order_value,
        opp.max_installments,
        opp.payment_type_count,
        ROUND(opp.total_payment_value/ NULLIF(ov.order_value, 0), 2)
AS payment_to_order_ratio
FROM olist_orders_dataset o
JOIN order_payment_profile opp
     ON o.order_id = opp.order_id
JOIN order_value ov
     ON o.order_id = ov.order_id
WHERE o.order_status = 'delivered'
```

FLAGGING ABNORMAL PAYMENT PATTERNS

```
WITH payment_features AS (
     SELECT     o.order_id,
                COUNT(*) AS payment_attempts,
            SUM(p.payment_value) AS total_payment_value,
            MAX(p.payment_installments) AS max_installments,
            COUNT(DISTINCT p.payment_type) AS payment_type_count
    FROM olist_orders_dataset o
    JOIN olist_order_payments_dataset p
           ON o.order_id = p.order_id
     WHERE o.order_status = 'delivered'
    GROUP BY o.order_id
),

order_value AS (
     SELECT     order_id,
                SUM(price + freight_value) AS order_value
    FROM olist_order_items_dataset
```

```
    GROUP BY order_id
)

SELECT     pf.order_id,
           pf.payment_attempts,
           pf.max_installments,
           pf.payment_type_count,
        pf.total_payment_value,
        ROUND(ov.order_value, 2) AS order_value,
        ROUND(pf.total_payment_value/ NULLIF(ov.order_value, 0), 2)
AS payment_to_order_ratio,
        CASE
                WHEN   pf.payment_attempts   >=   3   THEN   'Multiple
Payments'
            WHEN pf.max_installments >= 10 THEN 'High Installments'
            WHEN pf.payment_type_count > 1 THEN 'Mixed Payment Types'
            WHEN pf.total_payment_value > ov.order_value * 1.2 THEN
'Overpayment'
            ELSE 'Normal'
           END AS anomaly_flag
FROM payment_features pf
JOIN order_value ov
     ON pf.order_id = ov.order_id
WHERE pf.payment_attempts >= 3
     OR    pf.max_installments >= 10
    OR     pf.payment_type_count > 1
    OR     pf.total_payment_value > ov.order_value * 1.2
ORDER BY anomaly_flag
```

KEY INSIGHT

The Payment Profile and Abnormal Flags resulted in a small subset of orders exhibits abnormal payment behaviour. Most orders follow simple payment patterns but a small fraction show excessive installment usage, multiple payment attempts, and mixed payment methods. Payment amount mismatch is a strong risk signal. Orders where total payments significantly exceed order value indicate payment retries, system inconsistencies, and potential fraud or dispute scenarios. Complex payment structures increase operational risk. Orders with many installments or multiple payment methods are harder to reconcile and more costly to support. Abnormal does not necessarily mean fraud, but requires attention. These patterns should not be auto-blocked, but prioritized for review.

BUSINESS RECOMMENDATION

1. Introduce Real-Time Payment Anomaly Monitoring
    - Flag orders at checkout or post-payment
    - Route high-risk orders for automated review
2. Set Soft Limits on Payment Complexity
    - Cap installment counts dynamically based on order value
    - Limit mixed payment types for low-trust users
3. Align Payment Flexibility with Risk Profiles
    - More flexibility for trusted Customers or High-SLA Sellers
    - Less flexibility for new or risky segments
4. Feed Payment Signals into Risk Scoring Models
    - Combine Payment Anomalies, Delivery Performance, and Review History into a unified fraud/ risk score.